

七、安全与权限：Agent 的行为边界

核心问题： 如何确保 AI Agent 在代码库中的操作安全可控？

OODA 定位： 全环节的安全底线——不是优化循环的某个环节，而是给整个循环画边界

上一节结尾的问题：当 Agent 可以 spawn 子 Agent、并发修改文件、执行 shell 命令时，如何确保它不会搞出灾难？

答案回到第二讲的核心论点：**结构化工具集是精细权限控制的前提。**

第二讲比较了四个流派时说过——harness 看到 `Read(file="secret.key")` 可以拒绝、看到 `Bash(command="rm -rf /")` 可以拦截——**工具名本身就是权限标签**。但面对一段 Python 代码，你在执行前无法静态分析它是读文件还是删文件。CC 的权限模型（Read 允许、Write 需确认、Bash 受限执行）在 CodeAct 架构下根本无法实现。

本节给出完整实证。

7.1 三道防线

第一道：工具级权限分级

CC 源码实证（`src/utils/permissions/permissions.ts` ~1487 行，`src/types/permissions.ts` ~442 行）

CC 定义了 7 种权限模式：

模式	行为	典型场景
<code>default</code>	危险操作需用户确认	日常使用
<code>acceptEdits</code>	文件编辑自动允许，shell 命令需确认	信任文件操作
<code>bypassPermissions</code>	跳过所有检查	完全信任
<code>dontAsk</code>	需确认的操作自动拒绝	CI/CD 无人值守
<code>plan</code>	计划模式，需审批后执行	敏感项目

模式	行为	典型场景
auto	AI 分类器判断操作是否安全	规则覆盖不到时的兜底
bubble	将权限请求冒泡给父 Agent	子 Agent（第六讲已介绍）

工具被分为两类：

自动允许（只读操作，不修改文件系统）：
Glob, Grep, Read, LSP, WebSearch, TaskCreate/Get/Update/List

需要确认（写入/执行，可能产生副作用）：
Bash（非只读命令）, Edit, Write, NotebookEdit, Agent（特定模式）

这里“自动允许”是 CC 的默认策略——Read 可以读任意路径（包括项目外的文件），这在企业场景下可能需要通过 Managed 层的 CLAUDE.md 或 deny 规则进一步限制。CC 额外对 .git/、.claude/ 等安全路径做了硬编码保护。

这个分类之所以可行，正是因为每次调用都是一个命名的操作 + 结构化参数。harness 可以对任意维度做规则匹配——工具名、参数内容、文件路径。CodeAct 的一段代码片段无法提供这种粒度。

第二道：内容级规则 + 8 个规则来源

CC 的权限不仅按工具名分级，还支持内容级匹配：

规则示例：

Bash(git *) → 允许所有 git 命令

Bash(npm publish:*) → 需要确认

Edit(/etc/*) → 拒绝（系统文件不可编辑）

8 个规则来源形成层级覆盖：

userSettings → projectSettings → localSettings → flagSettings
→ policySettings → cliArg → command → session

决策流水线（hasPermissionsToUseToolInner()，行 1158）分三步：

Step 1: Deny/Ask 规则检查

→ 工具是否被 deny rule 禁止？

→ 内容级规则是否匹配？

→ 安全路径检查（.git/, .claude/ 等）

Step 2: 模式决策

- `bypassPermissions` 直接允许
- 检查 `always-allow` 规则

Step 3: 默认 `passthrough` → 提示用户

`auto` 模式值得单独说明：当规则覆盖不到时，CC 会用**模型本身**来判断一个操作是否安全——本质上是一个安全分类器。

CC 源码实证（`cli.js`，`Em4()` 函数）：`auto` 模式的 AI 分类器有几个工程细节值得注意：

1. **分类器使用用户的活跃模型**（不是独立的小模型）——即如果主循环用 Opus 4.6，分类器也用 Opus 4.6。可以通过 `tengu_auto_mode_config` 配置覆盖
2. **两阶段分类**：第一阶段用极少的 token（快速模式 256 token，普通模式仅 64 token）做快速判断——大多数明显安全/危险的操作在此阶段就能决策；只有不确定的操作才进入第二阶段（4096 token 深度分析）
3. **分类格式是结构化 XML**——`xml_2stage`、`xml_fast`、`xml_thinking` 三种变体，全部输出结构化的安全评估

这再次说明结构化工具集的价值：分类器的输入是 `toolName + structuredParams`（如 `Bash(command="git push --force")`），可以精确评估。如果输入是一段任意 Python 代码，分类器需要做完整的代码语义分析才能判断安全性——这在 64 token 的 budget 内不可能完成。

这是“用 AI 管 AI”的实践，有其局限（分类器也可能误判），但提供了规则之外的灵活兜底。

第三道：执行约束

注意：“沙箱”在这里不是指 Docker 那种虚拟环境，也不是模拟执行——命令是**真实执行**的，`harness` 只是给执行加了安全围栏：

- **超时限制**：默认 2 分钟。防止 `while true`、`npm install` 卡死等失控进程
 - **破坏性操作拦截**：`git push --force`、`rm -rf`、删除分支等高风险操作，即使在自主模式（`bypassPermissions`）下也需要用户确认
 - **安全路径保护**：`.git/`、`.claude/` 等关键目录的文件受额外保护
-

7.2 Hook 系统：harness 层面的事件触发器

CC 源码实证 (src/query/stopHooks.ts)

Hook ≠ Tool。这是一个重要区分：

- **Tool** 是 action space 的一部分——模型决定什么时候调用、传什么参数
- **Hook** 是 harness 层面的事件触发器——模型完全不知道 hook 的存在，也不能主动调用它

Hook 是用户预配置的 shell 命令，由 harness 在特定事件发生时**自动触发**：

用户配置 (settings.json) :

```
"hooks": {
  "afterToolUse": [{ "matcher": "Edit", "command": "eslint $FILE" }]
}
```

执行流程：

模型调 Edit(file="app.js", ...)

→ harness 执行 Edit (真实修改文件)

→ harness 自动跑 eslint app.js (Hook 触发，模型不知情)

→ eslint 通过 → 模型看不到任何反馈

→ eslint 失败 → harness 将错误信息注入对话历史

→ 模型看到的是 "Your edit introduced a syntax error: ..."

三种 hook 返回结果：

返回类型	harness 行为	场景
blocking error	注入错误消息，强制模型继续修正	lint 失败、测试未通过
non-blocking feedback	信息性输出，不影响循环	日志记录、统计
阻止继续	终止循环	安全策略触发紧急停止

权限引擎决定 “能不能做”，Hook 实现 “做了之后自动检查什么”。两者配合形成完整的安全策略链：先过权限关，再过 Hook 关。

7.3 CLAUDE.md：自然语言 “宪法”

CC 源码实证 (src/utils/claudemd.ts)

权限引擎处理的是结构化规则（允许/拒绝/确认），但很多行为约束无法用规则表达——“修改 auth 模块时同步更新 middleware”、“不要用自定义脚本做数据库迁移，用 pg_dump”。

CLAUDE.md 用**自然语言**定义这些约束。CC 按 5 层顺序加载：

1. Managed:	/etc/claude-code/CLAUDE.md	← 企业全局策略
2. User:	~/.claude/CLAUDE.md + rules/*.md	← 个人偏好
3. Project:	CLAUDE.md, .claude/CLAUDE.md	← 项目约定 (checked in)
4. Local:	CLAUDE.local.md	← 本地覆盖 (不 checked in)
5. AutoMem:	MEMORY.md	← 自动记忆

回扣第三讲的 Reflexion：CLAUDE.md 不仅是行为约束，也是**显式的 Reflexion Memory**——人类将失败经验编码为自然语言，持久化存储，供 Agent 在每次会话中参考。只不过 Reflexion 论文中 Memory 由模型自己写，CLAUDE.md 由人类写。

5 层加载的设计意图：**从全局到局部，从组织到个人，层层覆盖**。企业可以在 Managed 层禁止某些操作，项目可以在 Project 层定义编码规范，个人可以在 Local 层覆盖不影响团队的偏好。这本质上是一个**自然语言的配置管理系统**。

7.4 本节小结

三层安全机制：

权限引擎 → "能不能做"
7 种模式 × 8 个规则来源 × 内容级匹配
~1500 行 TypeScript，结构化工具集是前提
Hook 系统 → "做了之后自动检查什么"
harness 层面的事件触发器，模型不可见
lint、测试、安全扫描—自动化的质量门禁
CLAUDE.md → "应该怎么做"
自然语言行为约束，5 层加载
项目约定 + 显式 Reflexion Memory

结构化工具集是这一切的前提。第二讲说“工具名本身就是权限标签”——本节用 CC 的 ~1500 行权限引擎证实了这个论点。如果 Agent 的动作是任意代码（CodeAct），你无法在执行前判断它会做什么，精细权限控制无从谈起。

加餐预告：六讲手工拆解了 Agent 的引擎盖——system prompt、工具接口、上下文管理、循环控制、协同编排、安全边界。这些设计目前都是人类工程师手工调优的。但最新的研究表明：Harness 本身也可以被自动优化。接下来的加餐环节，我们看看 Meta-Harness 论文如何用 `coding agent` 来搜索优化另一个 agent 的 harness code，然后做一个完整的双主线回顾。